中山大學
SUN YAT-SEN UNIVERSITY

数据抽象和类II

中山大学计算机学院

主讲人：万海

wanhai@mail.sysu.edu.cn

C/C++程序内存布局与对象（变量）存放位置

正文

- 函数实现，库实现，字符串等资源。不可改

静态变量（对象）

- main 执行前初始化数据
- 用零初始化

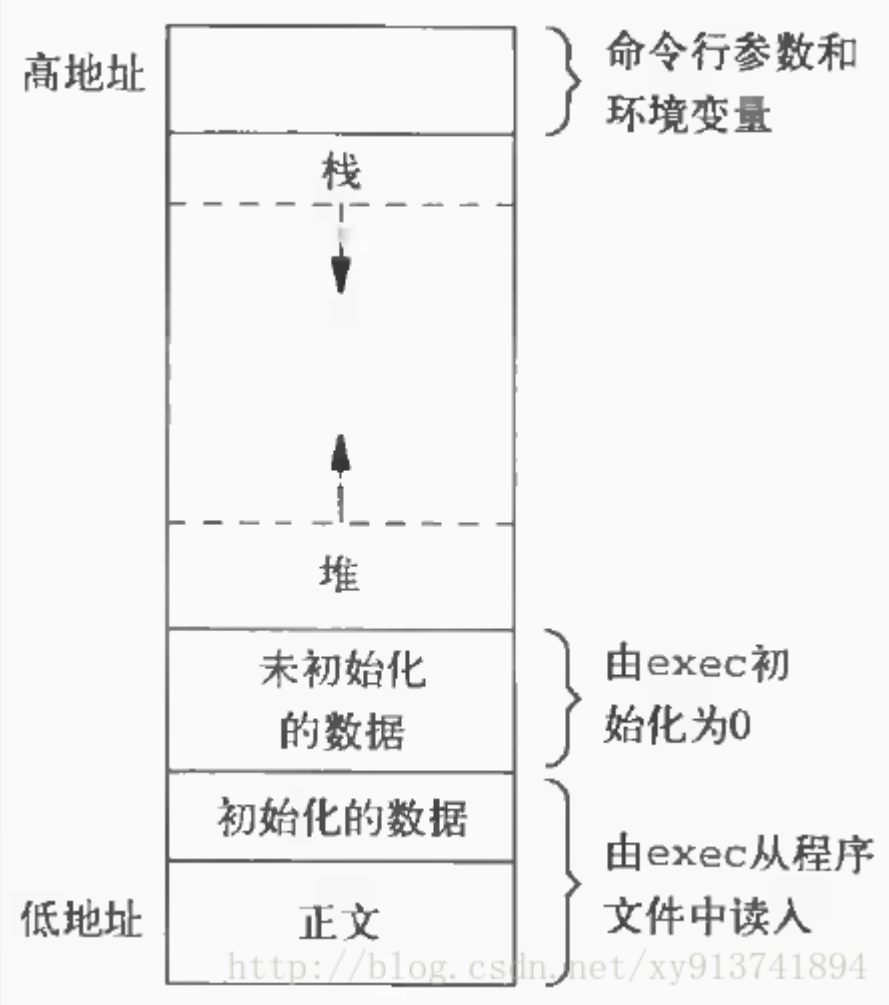
栈（Stack）

- 函数参数，**自动变量**（对象）

堆（Heap）

- **动态变量**（对象），由 stdlib.h 管理

```
char *s1 = "Literal";  // 文字在代码区，仅分配了字符指针
// s1[0] = 'I';  //Segmentation fault
char s2[] = "Initial Literal"; // 分配数组空间
Point *po = &((Point){2,3}); // 分配结构空间且指针
```



C语言动态对象（变量）管理

堆（Heap）

- 共享的对象（变量）空间
- 由 stdlib 库管理

正确使用堆空间

- 必须 #include <stdlib.h>
- 申请空间，void * malloc(size_t)
- 释放空间，void free(void *)
- 申请的空间必须释放，否则就是**内存泄漏**
- Free 后**再使用指针或释放**，行为不可预测
- 如何申请、释放对象或一维数组？
- 如何申请、释放二维数组？

C++也是这样用的吗？

```c
#include<stdio.h>
#include<stdlib.h>

typedef struct { int x; int y; } Point;

int main() {
    // 分配变量或一维可变数组
    Point *p1 = malloc(sizeof(Point) * 10);
    // 分配二维数组(n,m是常数)
    Point (*p2)[2][3] = malloc(sizeof(Point) * 6);
    // 分配数组的数组
    int n = 2, m = 3;
    Point **p3 = malloc(sizeof(Point *) * n);
    for (size_t i = 0; i < n; i++)
    {
        p3[i] = malloc(sizeof(Point) * m);
    }
    // do somesthing
    free(p1);
    free(p2);
    // 必须先释放行数组
    for (size_t i = 0; i < n; i++)
    {
        free(p3[i]);
    }
    free(p3);
    return 0;
}
```

C++用 malloc 分配对象数组（超纲）

程序要点

- #include<cstdlib>
- if 语句判断空间是否分配
- 初始化对象
  - new(void *) 构造函数
- 处理构造异常
- 调用对象析构
  - 对象.~析构函数
- 最后释放空间

```
#include <iostream>
#include <cstdlib>
#include <string>

int main() {
    // 为 4 个 string 的数组分配足够空间
    if(auto p = (std::string*)std::malloc(4 * sizeof(std::string))) {
        int i = 0;
        try {
            for(; i != 4; ++i) // 填充数组
                new(p + i) std::string(5, 'a' + i);

            for(int j = 0; j != 4; ++j) // 打印出来
                std::cout << "p[" << j << "] == " << p[j] << std::endl;
        }
        catch(...) {}

        for(; i != 0; --i) // 清理
            p[i - 1].~basic_string();

        std::free(p);
    }
}
```

C++新关键字：new 和 delete 运算符

新关键字优点：

- **new** 类型 初始化
  - 分配空间
  - 每个对象调用构造器
  - 有错误抛出异常
- **delete []p**
  - 为数组每个对象析构
  - 释放空间

Cppreference：new 表达式

```
/*new-delete-strings-demo*/
#include <iostream>
#include <string>

int main() {
    // 为 n 个 string 的数组分配空间
    int n = 4;
    std::string *p = new std::string[n] {
        std::string(5, 'a'),
        std::string(5, 'b'),
        std::string(5, 'c'),
        std::string(5, 'd')
        };

    for(int i=0; i<n; i++) {
        std::cout << p[i] << std::endl;
    }

    delete []p;
}
```

目录

CONTENTS

01 动态对象与指针

02 =delete 和 =default

03 对象成员初始化

04 对象的内存布局

05 拷贝构造函数

06 深入 const

动态对象（变量）与对象指针

动态对象的使用：new_object.cpp

- 申请
  - new 类型/构造函数
  - 返回该类型的指针
  - 不是万能指针 void *
- 使用
  - *p 解析为对象
- 释放
  - delete 指针

```
#include<iostream>
using namespace std;

int main() {
    int *p;
    p = new int;

    cout << "Please enter an integer value: ";
    cin >> *p;
    cout << "The value you enter is: " << *p;

    delete p;
}
```

```
Please enter an integer value: 3
The value you enter is: 3
```

动态对象（数组）与对象指针

动态对象的使用：new_array.cpp

- 申请数组
  - new 类型[]{初始化列表}
  - 在堆上分配对象数组并构造初始化。没有初始化列表调用无参构造（默认构造一般是不确定值）；常数初始化列表，后面补零；长度小于初始化序列长度，抛出异常
  - 返回该类型的指针
- 使用
  - *p 直接看作数组使用
- 释放
  - delete []指针

```
#include<iostream>
using namespace std;

void inc(int *p, int Length) {
    for (int i = 0; i < Length; i++)
        p[i]++;
}

int main() {
    int *p;
    int Length, i;

    cout << "Enter the lenght you want: ";
    cin >> Length;
    p = new int[Length]{1,3,5};

    inc(p, Length);

    for (i = 0 ; i < Length; i++)
        cout << *(p+i) << " ";

    delete []p;
}
```

new运算符的使用

指针=new 类型名；//动态创建一个对象

指针=new 类型名（初始化参数）；//动态创建一个对象并指定初始化参数

指针=new 类型名[数组长度]；//用于动态分配数组

- 初始化参数及其括号为可选
- 类型可为基本类型，也可为类类型，若为类类型，则初始化参数相当于将实际参数传递给该类的构造函数
- new运算返回一个该类型指针（不是 viod*），指向分配到的内存空间
- 若内存分配失败，抛出异常，不是 NULL

对象指针访问成员 - 例子

```
class Text {
public:
     Text(const char* s);
     ~Text();
     void Print();
private:
    string s;
};

Text::Text(const char* s){
    this->s = string(s); //在栈上创建一个临时对象
    cout <<"Creating " << this->s << endl;
}

Text::~Text() {
    cout <<"Releasing " << this->s << endl;
}
//using_objects.cpp
```

例子

```
int main() {
    Text* str_ptr;
    Text  str("on stack 1");

    str_ptr = &str;

    str.Print();
    str_ptr->Print();
}
```

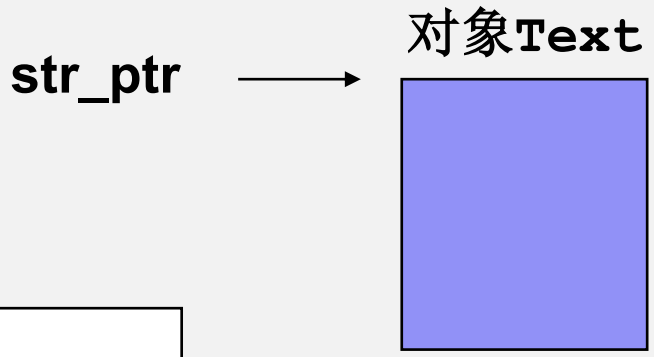
取对象地址

. 对象取成员

-> 指针取成员

```
Creating on stack 1
print on stack 1
print on stack 1
Releasing on stack 1
```

注意：
栈上对象
自动析构



对象的动态创建和撤销

```
//using object dynamic.cpp

int main() {
    Text* str_ptr;
    str_ptr = new Text("on heap 1");
    //调用构造函数会在栈上分配一个临时对象
    Text("on stack 1");

    str_ptr->Print();

    //delete str_ptr; //重要，堆空间对象不会自动析构
}
```

```
//using objects dynamic.cpp

int main() {
    Text* str_ptr = new Text[3]{"on heap 1","on heap 2","on heap 3"};

    Text* p = str_ptr;
    p++->Print();
    p->Print();

    delete []str_ptr; //重要，数组对象析构
    //delete str_ptr; 错误
}
```

delete操作符的使用

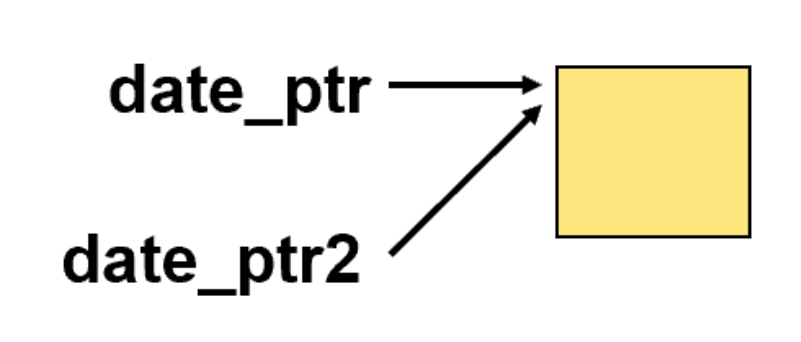
delete 变量名；//基本用法

delete []变量名；//用于释放数组

- 如果动态分配了一个数组，但是却用**delete p**的方式释放，没有用**[]**，则编译时没有问题，运行时可能不会发生错误，但实际上会导致动态分配的数组没有被完全释放。
- **delete**释放的是指针所指对象占据的内存。
- 用**delete**释放空间后，指针的值仍是原来指向的地址，但指针已无效（重复释放将出错）。
- **delete**对象指针，会调用该对象的析构函数。

非法（野）指针使用错误

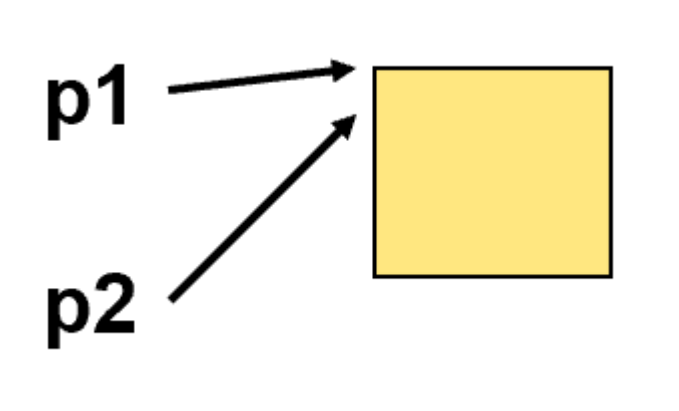
```
DATE* date_ptr;
DATE* date_ptr2;
date_ptr = new DATE(1976, 12 ,20);

date_ptr2 = date_ptr;

date_ptr -> Print();     //正确
date_ptr2 -> Print();    //正确

delete date_ptr;       //将使date_ptr和date_ptr2指向失效的空间

date_ptr -> Print();     //错误，date_ptr已失效
date_ptr2 -> Print();   //错误，date_ptr2也已失效

delete date_ptr2;        //错误，不能再次释放
```



//invalid_ptr1.cpp

非法（野）指针使用错误

```
// invalid_ptr2.cpp
//对于指向简单数据类型的指针，也有相同的问题。
int* p1;
int* p2;

p1 = new int;
p2 = p1;

*p1  = 10;
cout << *p2 << endl;

delete p1;       //将使p1r和p2同时失效

cout << *p1;     //错误，p1已失效
cout << *p2;     //错误，p2也已失效

delete p2;        //错误，不能再次释放
```



内存泄漏：常见情况

内存泄漏：指 new 的空间失去了指针或引用永远无法释放。导致资源耗尽！

指针赋值时可能发生：

```
int* p = new int(7); // 动态分配的 int 带值 7
p = nullptr; // 内存泄漏
```

指针离开作用域：

```
void f()
{
    int* p = new int(7);
} // 内存泄漏
```

异常导致程序或函数中止：

```
void f()
{
    int* p = new int(7);
    g();        // 可能抛出异常
    delete p; // 若无异常则 ok
} // 若 g() 抛出异常则内存泄漏
```

显式定义默认函数：=default (C++11)

```
#include <iostream>
using namespace std;

class A
{
public:
    A(int x) {
        cout << "This is a parameterized constructor";
    }
    A() = default;
};

int main(){
    A a;            //call A()
    A x(1);         //call A(int x)
    cout<<endl;
    return 0;
}
```

令编译器为某个类生成特殊成员函数或比较运算符的显式指令。（C++11）

- 当我们声明有参构造函数时，编译器不会创建默认构造函数。
- =default 必须在函数申明后，让编译创建该构造函数
- 特殊成员函数包括：
  - 默认构造函数:T()
  - 析构函数:~T()
  - 复制构造函数:T(const T&)
  - 赋值运算:operator=(const T&)
  - …
- 函数不能再自定义实现

=default

= default 仅要用于编译器能自动生成的特殊成员函数

```
class B
{
public:
    // Error, func is not a special member function.
    int func() = default;

    // Error, constructor B(int, int) is not a special member function.
    B(int, int) = default;

    // Error, constructor B(int=0) has a default argument.
    B(int = 0) = default;
};
```

弃置函数：=delete (C++11)

```
class A {
public:
    A(int x): m(x) { }

    // Delete the copy constructor
    A(const A&) = delete;
    // Delete the copy assignment operator
    A& operator=(const A&) = delete;
    int m;
};

int main() {
    A a1(1), a2(2), a3(3);
    // Error, the usage of the copy assignment operator is disabled
    a1 = a2;
    // Error, the usage of the copy constructor is disabled
    a3 = A(a2);
    return 0;
}
```

如果取代函数体而使用特殊语法 = delete ;，则该函数被定义为弃置的（deleted）。

任何弃置函数的使用都是非良构的（程序无法编译）。这包含调用，包括显式（以函数调用运算符）及隐式（对弃置的重载运算符、特殊成员函数、分配函数等的调用），构成指向弃置函数的指针或成员指针，甚或是在不求值表达式中使用弃置函数。

//code10

=delete

请注意，删除的函数是隐式内联的，这一点非常重要。删除的函数定义必须是函数的首次声明。

以下方法是将函数声明为已删除的正确方法：

```
class C {
public:
        C(C& a) = delete;
};
```

但是以下尝试声明删除函数的方法会产生错误：

```
// incorrect syntax of declaring a member function as deleted
class C  {
public:
    C();
};

// Error, the deleted definition of function C must be the first declaration of the function.
C::C() = delete;
```

对象成员初始化 -类的组合

- 由多个对象组合成新的对象，即类的组合
- 例如：射线类，射线是从 origin 开始并按照某个 direction 行进的无限长的线。

```
struct Ray {
    Ray (Vector3 origin, Vector3 direction); //构造函数

    Vector3 origin;          //射线的原点。
    Vector3 direction;       //射线的方向。
}
```

- 问题：创建 ray 对象时如何初始化成员 origin 和 direction
  - C++11以前，只能在构造函数中使用 this->origin = Vector3(origin);
  - 这种方法先构造临时对象，再赋值，再释放临时对象，比较低效

对象成员初始化 - 成员初始化器（C++11）

- 类的非静态数据成员可以用下列几种方式之一初始化：
  - 在构造函数的成员**初始化器列表**中。（C++11）
  - 通过**默认成员初始化器**，它是成员声明中**包含的花括号**或**等号初始化器**。（C++11）
  - 构造函数体内进行赋值操作。（注意：**无法构造成员**，除非你特别熟悉 new 的各种用法）
- 为什么需要初始化器列表和默认成员初始化器？
  - 对象初始化分两个阶段：首先按**声明顺序**初始化成员、**然后**执行构造函数**函数体**；
  - 对于复杂对象成员，如 std::string name，必须先构造才能在构造函数中赋值。这会导致构造函数必须先构造一个 string 中间变量才能赋值给 name；
  - 对于**引用类型成员**，**const 成员**需要预初始化；
  - 对象成员**初始化需要参数**。
- 为了提升初始化性能，C++引入了默认成员初始化器和初始化器列表

成员初始化器列表（C++11）- 语法

构造函数名字、参数列表和函数体之间，可选初始化器列表。初始化器列表以冒号开头，后跟一系列以逗号分隔的成员初始化器，如成员初始化构造函数、该类的其他构造函数。

```
//member_initializer_lists.cpp
struct Ray {
    Ray (Vector3 origin, Vector3 direction):
       origin(origin),direction(direction) {};

    Vector3 origin;     //射线的原点。
    Vector3 direction;       //射线的方向。
};
```

构造函数与成员初始化器列表 - cppreference.com

成员初始化器列表 - 要点

必须使用初始化器列表的时候

除了性能问题之外，有些时候合初始化器列表是不可或缺的，以下几种情况时必须使用初始化器列表：

1. 常量成员，因为常量只能初始化不能赋值，所以必须放在初始化器列表里面
2. 引用类型，引用必须在定义的时候初始化，并且不能重新赋值，所以也要使用初始化器
3. 没有无参构造函数的类成员，使用初始化器列表可以使用有参构造函数初始化

成员初始化器列表 - 要点

成员变量声明的顺序

成员是按照他们在类中声明的顺序进行初始化的，而不是按照他们在初始化器列表出现的顺序初始化的：

```
class foo {
public:
      int i ;int j ;
      foo(int x):i(x), j(i){}; // ok，先初始化i，后初始化j
};
```

成员初始化器列表 - 要点

成员变量声明的顺序

再看下面的代码:

```
class foo
{
public:
      int i ;int j ;
      foo(int x):j(x), i(j){} // j 值未定义
};
```

这里i的值是未定义的因为虽然j在初始化器列表里面出现在i前面，但是i先于j定义，所以先初始化i，而i由j初始化，此时j尚未初始化，所以导致i的值未定义。一个好的习惯是，按照成员声明的顺序进行初始化。

对象成员初始化器与初始化器列表 - 例子

```
/*member initialization*/
struct S {
    int n;                      // 非静态数据成员
    int& r = n;                 // 引用类型的非静态数据成员; =默认构造
    int a[2] = {1, 2};          // 带默认成员初始化器的非静态数据成员（C++11）
    std::string s{'H','C'};     // 带默认成员初始化器
    struct NestedS  {
          std::string s;
          NestedS(std::string s = "hello"):s(s) {};    //构造函数
    } d5;                       // 具有嵌套类型的非静态数据成员

    const char bit : 2;         // 2 位的位域, const 初始化

    S():n(7),bit(3) {}          // ":n(7),bit(3)" 是初始化器列表; "{}" 是函数体
    S(int x):n{x},bit(3) {}
};

int main()
{
    S s;       // 调用 S::S()
    S s2(10); // 调用 S::S(int)
}
```

对象成员初始化 - 练习

- 修改 S() 函数，在其中输出成员 r 的值，请解释结果
- 修改成员 d5 的构造函数输出嵌套成员 s 的值，请问先输出 r 还是 d5.s ？
- 修改 S()， 请输出成员 s 的值，请解释结果
- 请修改成员 s 的初始值，使用 std::string(4,'C') 初始化 s
- 请修改d5.s 的初始值为 "world", 请给出至少两种初始化 d5 的方法
- 修改 S() 的成员初始化列表，删除 bit(3)，请解释结果
  - 在构造函数内给 bit 赋值，可行吗?
  - 在声明中给出默认初始化器，可行吗?

练习：Stack 的实现

阅读 stack.cpp 关于 栈类 的实现，回答以下问题：

- nullptr 关键字是指针类型空常量
  - 与 NULL 常量有区别吗?
- 知识点及应用
  - Stack 的哪些成员函数使用了 new 和 delete 操作， new-delete 操作配对吗?
  - 如果不使用 =delete 弃用复制初始化和赋值，会出现非法指针吗？请修改代码，证明指针错误使用
  - 哪里使用了默认初始化器，可以使用其他方法初始化成员 head 吗?
- 对象与封装
  - 在客户端角度，使用 Stack 需要指针、链表等知识吗？为什么?
  - 请写一个用数组实现的 Stack 类，保证客户端不用修改哦!

中山大學
SUN YAT-SEN UNIVERSITY

谢谢

中山大学计算机学院

中山大学MOOC课程组